

OOP 期末历年卷整理

资料范

围： final/18-19.pdf 、 final/19-20春夏/*.pdf 、 final/23-24春夏.pdf 、 final/24-25秋冬.html 。

说明：18-19 和 19-20 中有几份是图片式 PDF，本文对清晰可读部分做考点归纳；23-24、24-25 可完整抽取文本，是主要依据。

一、考试画像

年份/ 资料	题型结构	明显考点
2018- 2019	判断 10、单选 10、 结果填空 9、 程序填空 3	异常、流、inline、运算符重载、namespace、vector、构造/ 析构、继承、模板、链表、Fraction、字符数组
2019- 2020 春夏	程序填空、 结果填空、 程序设计大题	polymorphic linked list、Fraction 运算符重载、泛型循环队列、 构造/析构输出、拷贝、异常、继承、虚函数
2023- 2024 春夏	单选 10、结果填空 6、程序填空 2、 主观程序设计 1	拷贝构造、多级/多继承、static_cast/dynamic_cast 、 默认构造、友元重载限制、异常析构、模板特化、前/后置 ++ 、namespace、模板容器、inner_product 、Shape 多态
2024- 2025 秋冬	单选 15、结果填空 8、程序填空 2、 主观程序设计 1	inline、运算符重载、friend、拷贝构造、析构、虚函数/ 对象切片、纯虚函数、override、异常传播、模板、static 成员、this 、默认参数、引用、动态数组、责任链模式

结论：这门课期末不是只背概念。高分核心是能稳定手推 C++ 对象生命周期、虚函数绑定、异常传播、模板实例化、运算符重载和小型类设计。

二、高频考点优先级

S 级：几乎每年出现

1. 构造、拷贝构造、赋值、析构

- 什么时候调用拷贝构造：按值传参、按值返回、用同类对象初始化新对象。
- 什么时候调用赋值：已有对象之间 `a = b`。
- 析构顺序：派生类析构先执行，再执行基类析构；成员对象通常逆构造顺序析构。
- 基类指针删除派生对象：基类析构函数必须是 `virtual`，否则派生析构可能不被调用。
- 对象切片：`Base b = derived;` 或 `*basePtr = derived;` 只保留 Base 子对象，之后虚调用也按对象真实类型 Base 来。

2. 继承、多态、虚函数

- `virtual` 决定运行时绑定；非虚函数看静态类型。
- `override` 要求函数名、参数、`const` 等签名完全匹配基类虚函数。
- 纯虚函数 = 0 使类成为抽象类；派生类必须实现所有未实现纯虚函数才能实例化。
- `public` 继承下，基类 `public` 成员在后续多级继承中继续作为 `public` 成员继承。

3. 异常处理

- `throw` 会离开当前控制流，沿调用栈找匹配 `catch`。
- `catch (T x)` 像函数参数，可以访问抛出的值；不需要值时可写 `catch (...)` 或某些场景省略变量名。
- `throw;` 只能在 `catch` 中重新抛出当前异常。
- 异常展开会析构已经构造完成的自动对象；动态分配对象需要自己 `delete` 或使用智能指针。

4. 运算符重载

- 重载不改变优先级、结合性、操作数个数。
- `=`, `[]`, `()`, `->` 必须作为成员函数重载，不能作为普通友元函数重载。
- 前置 `++x` 常返回引用；后置 `x++` 多一个 `int` 哑参数，通常返回旧值副本。
- 输出运算符一般写成非成员/友元：`friend ostream& operator<<(ostream&, const T&);`

5. 模板与 STL

- 函数模板：`template <class T> T func(T a)。`
- 类模板特化：`template <> class A<int> { ... };`
- 偏特化：`template <typename T> class MyClass<T*> { ... };`
- 迭代器区间常用 `[first, last)`，手写算法要记得同步递增多个迭代器。

A 级：稳定高频

1. `static_cast` 与 `dynamic_cast`

- `dynamic_cast` 用于安全向下转型时，基类通常必须是多态类，即至少有一个虚函数。
- 转指针失败返回 `nullptr`；转引用失败抛 `bad_cast`。

- `static_cast` 编译期转换，不做运行时类型检查；无继承关系的类指针通常不能互转。

2. static 成员

- static 数据成员属于类，不属于某个对象。
- 通常类外定义：`int Counter::count = 0;`
- static 成员函数没有 `this` 指针，不能直接访问非 static 成员。

3. namespace 与重载解析

- 同名函数可以由 namespace 区分。
- `using namespace A; using namespace B;` 后调用 `func(a)` 仍会按参数类型做重载解析。

4. 引用、this、默认参数

- 引用是变量别名；常见考法是“引用和原变量地址相同”。
- `this` 指向当前对象，只在非 static 成员函数中存在。
- 默认参数写在参数列表中，通常放声明处。

5. inline 与 friend

- inline 主要减少函数调用开销，不是为了缩短代码空间。
- 含循环、递归、static 局部变量等复杂函数通常不适合 inline。
- friend 函数不是成员函数，但可访问类的 `private/protected`；它可以是另一个类的成员，也可以是普通函数。

B 级：会出，但通常和大题结合

1. 动态内存与 Rule of Three

- 类中有 `new[]`，基本就要写析构、拷贝构造、赋值运算符。
- 赋值运算符要处理自赋值：`if (this != &other)`。
- 析构释放数组用 `delete[]`，释放单对象用 `delete`。

2. 流与文件

- 插入器 `<<` 可输出基础类型和指针等。
- manipulator 如 `endl`、`setw`、`setprecision` 可插入/抽取到流中。
- 大题可能要求 `ifstream` 解析文本并创建不同派生类对象。

3. 设计模式/类层次设计

- 23-24 是 Shape 文档解析。
- 24-25 是 Chain of Responsibility 责任链。
- 本质都是：抽象基类 + 虚函数接口 + 派生类实现 + 指针/容器管理对象。

三、按题型整理

1. 单选/判断题

常考结论：

- 只定义了有参构造函数，编译器不会再隐式生成默认构造函数。
- 构造函数不能声明为 `virtual`；析构函数经常需要声明为 `virtual`。
- OOP 语言要求对象、类/类型、封装状态、接口操作、继承等；“算法是基本逻辑构件”更偏过程式。
- C++ 支持面向对象、过程式、泛型；对声明式支持不是主要特色。
- 多继承写法： `class D : public B1, public B2 { };`
- 友元函数不一定是另一个类成员。
- 运算符重载不改变优先级和结合性。
- `operator[]` 不能作为 `friend` 非成员重载。
- 引用不是指针，不用 `*` 解引用，常与原变量同地址。
- 纯虚函数由派生类实现；抽象基类不能直接实例化。

满分做法：看到概念题先判断它在问“语法规则”还是“运行时行为”。例如 `override` 是编译期签名检查；虚函数调用是运行时动态绑定；默认构造是否生成是编译器规则。

2. 结果填空/代码输出

固定手推流程：

1. 先标出所有对象创建点：普通局部对象、数组对象、`new`、函数参数、返回值临时对象。
2. 再标出拷贝点：按值传参、返回对象、初始化、赋值。
3. 区分静态类型和动态类型： `Base* p = new Derived; p->virtualFunc();`
4. 遇到异常，画调用栈：抛出位置 -> 最近匹配 `catch` -> 是否重新抛出。
5. 遇到模板，先决定实例化版本：普通模板、全特化、偏特化。
6. 遇到前/后置 `++`，写出“返回谁、对象自己变成什么”。

代表题：

- 23-24 异常输出：
 - `FuncA(4)` 输出 `FuncA`
 - `FuncA(10)` 抛 `RangeError`
 - `FuncB` 的 `catch` 输出 `FuncB` 和 `ID:10`
 - 重新抛 `AnyError` 后 `main` 的 `catch (...)` 输出 `Main`
- 23-24 模板特化输出：
 - `A<double>` -> `A<double>`
 - `A<char>` -> `A<T>`
 - `A<int>` -> `A<int>`
- 23-24 前后置自增与 `+=`：
 - `t = w++`： `t` 得到旧 `w`， `w` 追加 `Ho`
 - `w += (t++)`： `+=` 用的是 `t++` 返回的旧值

- ++(...) 再追加 Hey
- 输出: HoHaHey
- 24-25 虚函数 + 对象切片:
 - *M = m; M->Define(); 输出 Mammal, 因为赋值到 Mammal 对象发生切片。
 - M = &m; M->Define(); 输出 Male, 因为指针指向真实 Male 对象且函数 virtual。
- 24-25 dynamic/static cast:
 - Base* b = new Derived(); dynamic_cast<Derived*>(b) 成功。
 - static_cast<int>(3.14159) 得到 3。

3. 程序填空题

常见空位:

- 构造函数初始化列表: Shape(const string& name) : mName(name) {}
- 虚析构: virtual ~Shape() {}
- public 继承: class Circle : public Shape
- 基类指针数组: Shape* shapes[2];
- 动态释放: delete shapes[i];
- 模板偏特化: template <typename T> class MyClass<T*>
- 迭代器算法:

```
template <class InputIt1, class InputIt2, class T, class BinaryOp1, class BinaryOp2>
T inner_product(InputIt1 first1, InputIt1 last1, InputIt2 first2, T init,
                BinaryOp1 op1, BinaryOp2 op2)
{
    while (first1 != last1) {
        init = op1(init, op2(*first1, *first2));
        ++first1;
        ++first2;
    }
    return init;
}
```

- 链式/分块数组:
 - 构造时 next = NULL 或 nullptr
 - T* data;
 - Array<T>* next;
 - 类外成员定义: T& Array<T>::operator[](int i)
 - 递归访问下一块: return (*next)[i - BLK_SIZE];

4. 主观程序设计题

23-24 Shape 文档解析大题套路：

- 抽象基类 shape 提供 `virtual void draw() = 0;` 和 `virtual bool parseattribute(istream&) = 0;`
- Rectangle/Circle/Ellipse 分别保存自己的坐标和尺寸。
- parseattribute 按题目固定文本格式读入关键字和数字。
- draw 按指定格式输出。
- CShapeDocument 用 `vector<shape*>` 管理多态对象，析构中逐个 delete 。

24-25 责任链大题套路：

- SupportQuery 封装 type，提供 `getType()`。
- Handler 是抽象接口，至少有：

```
virtual Handler* SetNext(Handler* handler) = 0;  
virtual string Handle(SupportQuery& query) = 0;  
virtual ~Handler() {}
```

- BaseHandler 保存 nextHandler，不能处理时转发。
- BillingHandler/TechnicalHandler/CustomerServiceHandler 判断 `query.getType()`，匹配则返回处理字符串，否则调用 `BaseHandler::Handle(query)`。

主观题拿满分的关键：

- 先写接口，再写派生类，不要一上来堆 main。
- 基类析构函数加 `virtual`。
- 返回类型、函数名、参数必须和题干完全一致。
- 输出字符串必须逐字匹配，包括大小写、空格、换行。
- 看到 new 就想清楚谁 delete；题目若给了 cleanup，也要保证析构虚。

四、必背代码骨架

1. 带动态数组的类

```
class DynamicArray {
public:
    DynamicArray(int n = 0) : size(n), data(new int[n]) {}

    DynamicArray(const DynamicArray& other)
        : size(other.size), data(new int[other.size]) {
        for (int i = 0; i < size; ++i) data[i] = other.data[i];
    }

    DynamicArray& operator=(const DynamicArray& other) {
        if (this != &other) {
            delete[] data;
            size = other.size;
            data = new int[size];
            for (int i = 0; i < size; ++i) data[i] = other.data[i];
        }
        return *this;
    }

    ~DynamicArray() { delete[] data; }

private:
    int size;
    int* data;
};
```

2. 抽象基类 + 多态

```
class Base {
public:
    virtual void f() = 0;
    virtual ~Base() {}
};

class Derived : public Base {
public:
    void f() override {}
};
```

3. 输出运算符重载

```
class X {
    friend ostream& operator<<(ostream& os, const X& x);
};

ostream& operator<<(ostream& os, const X& x) {
    return os;
}
```

4. 前置/后置自增

```
class X {
public:
    X& operator++() {
        // change *this
        return *this;
    }

    X operator++(int) {
        X old = *this;
        // change *this
        return old;
    }
};
```

5. 模板特化与偏特化

```
template <typename T>
class A {};
```

```
template <>
class A<int> {};
```

```
template <typename T>
class A<T*> {};
```

五、考前冲刺路线

第 1 轮：概念速通

必须能秒答：

- 构造/析构/拷贝构造/赋值的触发条件。
- 哪些运算符必须成员重载。
- virtual 、 override 、 纯虚函数、抽象类。
- static 成员、 this 指针、引用。
- static_cast 和 dynamic_cast 区别。
- inline、friend、namespace、默认参数。

第 2 轮：代码输出专项

每天练 20-30 分钟，只做手推：

- 构造析构输出顺序。
- 异常嵌套和重新抛出。
- 虚函数与对象切片。
- 模板特化输出。
- 运算符重载，尤其前置/后置 ++ 。
- 动态数组/指针浅拷贝与深拷贝。

第 3 轮：程序填空专项

重点背这些语法位置：

- `Class::function`
- `template <typename T>`
- `Array<T>::operator[]`
- 初始化列表 : `member(value)`
- `virtual ~Base()`
- `friend ostream& operator<<`
- `return *this`
- `delete[] data`
- `new T[n]`

第 4 轮：大题模板化

主观题通常不是让你发明复杂算法，而是让你把类体系写完整：

1. 抽象接口写对。
2. 派生类数据成员写对。
3. 构造函数初始化写对。
4. 虚函数 `override` 写对。
5. 输出格式写对。
6. 指针释放写对。

六、最容易丢分的坑

- 把拷贝构造和赋值运算符混淆。
- 基类析构忘记 `virtual`。
- `operator[]` 写成 `friend` 非成员。
- 后置 `++` 忘记 `int` 参数，或返回了修改后的对象。
- `dynamic_cast` 的基类没有虚函数。
- 类模板成员函数类外定义忘记 `Array<T>::`。
- `delete[]` 写成 `delete`。
- 输出题少算函数形参按值传递产生的临时/拷贝。
- 异常题忘记已构造局部对象会析构。
- 程序设计题输出字符串和题目不一致。

七、满分策略

选择判断题靠规则，代码输出题靠表格，程序填空题靠语法骨架，主观题靠接口复刻。

真正要拿满分，考场上每道代码题都在草稿纸上画三列：

Step	Event	Output
1	创建/拷贝/赋值/抛异常/虚调用	本步输出
2	进入 catch / 重新抛出 / delete	本步输出
3	离开作用域析构	本步输出

只要这张表不漏对象、不漏异常、不漏虚函数动态类型，OOP 期末的大部分分数都会很稳。

八、历年原题逐题摘录

这一部分按年份摘出原题。清晰文本卷尽量逐题分块；图片式卷子用 OCR 摘录，代码符号请以原 PDF 为准。

2024-2025 秋冬原题

单选题（每题2分） Q1

1. Inline functions are avoided when _____

- A. function contains static variables
- B. function have recursive calls
- C. function have loops
- D. all of the mentioned

答案：D

解析：

选 D。inline 适合短小、频繁调用的函数；有 static 局部变量、递归或循环时，编译器通常不会或不应内联，所以 all of the mentioned。

单选题（每题2分） Q2

2. Which is the correct statement about operator overloading?

- A. Only arithmetic operators can be overloaded
- B. Only non-arithmetic operators can be overloaded
- C. Precedence of operators are changed after overloading
- D. Associativity and precedence of operators does not change

答案：D

解析：

选 D。运算符重载只能改变这个运算符作用在自定义类型上时的行为，不能改变优先级、结合性、操作数个数。

单选题（每题2分） Q3

3. Pick out the correct statement.

- A. A friend function must be a member of another class
- B. A friend function cannot be a member of another class
- C. A friend function may or may not be a member of another class
- D. None of the mentioned

答案：C

解析：

选 C。friend 函数不是当前类的成员，但它可以是普通全局函数，也可以是另一个类的成员函数。

单选题（每题2分） Q4

4. When a copy constructor is called?

- A. When an object of the class is returned by value
- B. When an object of the class is passed by value to a function
- C. When an object is constructed based on another object of the same class
- D. All of the mentioned

答案：D

解析：

选 D。拷贝构造在“用同类对象创建新对象”时调用，也常出现在按值传参和按值返回中。

单选题（每题2分） Q5

5. When destructors are called?

- A. When a program ends
- B. When a function ends
- C. When a delete operator is used
- D. All of the mentioned

答案：D

解析：

选 D。析构发生在对象生命周期结束时：程序结束、函数局部对象离开作用域、或者对动态对象使用

delete。

单选题（每题2分） Q6

6. What will be the output of the following C++ code?

```
#include <iostream>
#include <string>
using namespace std;

class Mammal {
public:
    virtual void Define() {
        cout << "I'm a Mammal\n";
    }
};

class Human : public Mammal {
public:
    void Define() {
        cout << "I'm a Human\n";
    }
};

class Male : public Human {
public:
    void Define() {
        cout << "I'm a Male\n";
    }
};

class Female : public Human {
public:
    void Define() {
        cout << "I'm a Female\n";
    }
};

int main(int argc, char const *argv[]) {
    Mammal *M = new Mammal();
    Male m;
    Female f;
    *M = m;
    M->Define();
    M = &m;
    M->Define();
    return 0;
}
```

A. I'm a Male

I'm a Male

B. I'm a Male

I'm a Mammal

C. I'm a Mammal

I'm a Male

D. I'm a Mammal

I'm a Mammal

答案：C

解析：

选 C。*M = m 是把 Male 赋给一个 Mammal 对象，发生对象切片，所以第一次输出 Mammal；M = &m 后指针真实指向 Male，虚函数动态绑定，所以第二次输出 Male。

单选题（每题2分） Q7

7. Which is the correct statement about pure virtual functions?

A. They must be implemented inside a base class

B. Pure keyword should be used to declare a pure virtual function

C. Pure virtual function is implemented in derived classes

D. Pure virtual function cannot be implemented in derived classes

答案：C

解析：

选 C。纯虚函数让基类成为抽象类，派生类通常要实现它；题中“Pure keyword”不是 C++ 语法。

单选题（每题2分） Q8

8. Pick out the correct statement about the override.

A. Overriding refers to a derived class function that has the same name and signature as a base class virtual function

B. Overriding has different names

C. Overriding refers to a derived class

D. Overriding has different names & it refers to a derived class

答案：A

解析：

选 A。override 指派生类函数重写基类虚函数，函数名、参数列表、const 等签名必须匹配。

单选题（每题2分） Q9

9. In nested try-catch block, if the inner catch block gets executed, then_____

- A. Program stops immediately
- B. Outer catch block also executes
- C. Compiler jumps to the outer catch block and executes remaining statements of the main() function
- D. Compiler executes remaining statements of outer try-catch block and then the main() function

答案：D

解析：

选 D。内层 catch 执行后，如果没有继续 throw，控制流回到外层 try-catch 后续语句，再继续 main。

单选题（每题2分） Q10

10. What does this template function indicate?

```
template<class T>
T func(T a) {
    cout << a;
}
```

- A. A function taking a single generic parameter and returning a generic type
- B. A function taking a single generic parameter and returning nothing
- C. A function taking single int parameter and returning a generic type
- D. A function taking a single generic parameter and returning a specific non-void type

答案：B

解析：

选 B。函数声明为 T func(T a) 但函数体没有 return，语义上“输出 a，不返回有效值”；严格 C++ 中非 void 函数不返回值是不良写法，考试按“返回 nothing”处理。

单选题（每题2分） Q11

11. The static data member _____

- A. Can be accessed directly
- B. Can be accessed with any public class name

C. Can only be accessed with dot operator

D. Can be accessed using class name if not using static member function

答案： D

解析：

选 D。static 数据成员属于类；在普通成员函数里可直接访问，在类外通常用 `ClassName::member` 。

单选题（每题2分） Q12

12. What is the purpose of the this pointer in C++?

A. To refer to the current object within a member function.

B. To access static data members of the class.

C. To allocate memory dynamically for an object.

D. To initialize constant data members of the class.

答案： A

解析：

选 A。this 是非 static 成员函数里的隐含指针，指向当前调用该函数的对象。

单选题（每题2分） Q13

13. Which among the following is true for default arguments?

A. They are only allowed in the return type of the function declaration.

B. They are only allowed in the parameter list of the function declaration.

C. They are only allowed with the class name definition.

D. They are only allowed with the integer type values.

答案： B

解析：

选 B。默认参数写在函数参数列表中，通常放在函数声明处，从右往左连续给默认值。

单选题（每题2分） Q14

14. Which problem may arise if we use abstract class functions for polymorphism?

A. All classes are converted as abstract class

B. Derived class must be of abstract type

C. All the derived classes must implement the undefined functions

D. Derived classes can't redefine the function

答案： C

解析：

选 C。抽象基类有未定义的纯虚函数，具体派生类如果想实例化，就必须实现这些函数。

单选题（每题2分） Q15

15. Pick the correct statement about references in C++

- A. References stores the address of variables
- B. References and variables both have the same address
- C. References use dereferencing operator(*) to access the value of variable its referencing
- D. References were also available in C

答案： B

解析：

选 B。引用是变量别名，不是单独存放地址的指针；对引用取地址得到的是被引用对象地址。

填空题 Q1

1. What are the output of the following code?

```

#include <iostream>
using namespace std;

class BaseError {
public:
    BaseError() { cout << "BaseError" << endl; }
};

class FileError : public BaseError {
public:
    FileError(const string& filename) { m_filename = filename; }
    void display() { cout << "FileError: " << m_filename << endl; }
protected:
    string m_filename;
};

void ReadFile(const string& filename) {
    if (filename == "badfile.txt")
        throw FileError(filename);
    cout << "File read successfully" << endl;
}

void ProcessFile(const string& filename) {
    try {
        ReadFile(filename);
    } catch (FileError& fe) {
        fe.display();
        throw;
    }
}

int main() {
    try {
        ProcessFile("goodfile.txt");
        ProcessFile("badfile.txt");
    } catch (...) {
        cout << "Main exception caught" << endl;
    }
    return 0;
}

```

答案：

File read successfully [1分]

BaseError [1分]

FileError: badfile.txt [1分]

Main exception caught [1分]

解析：

先处理 goodfile，正常输出 File read successfully；再处理 badfile，构造 FileError 时先构造基类 BaseError，catch 中 display 输出文件名，然后 throw；重新抛出，被 main 的 catch(...) 捕获。

填空题 Q2

2. What are the output of the following code?

```

#include <iostream>
using namespace std;

class Shape {
public:
    virtual void draw() { cout << "Drawing a shape" << endl; }
    void display() { cout << "Shape" << endl; }
};

class Circle : public Shape {
public:
    void display() { cout << "Circle" << endl; }
};

class Square : public Shape {
public:
    void draw() override { cout << "Drawing a square" << endl; }
    void display() { cout << "Square" << endl; }
};

void render(Shape* shape) {
    shape->draw();
}

int main() {
    Circle circle;
    Square square;
    Shape shape;

    Shape* shapes[3];
    shapes[0] = &circle;
    shapes[1] = &square;
    shapes[2] = &shape;

    for (int i = 0; i < 3; ++i) {
        render(shapes[i]);
    }

    return 0;
}

```

答案：

Drawing a shape [1分]

Drawing a square [1分]

Drawing a shape [1分]

解析：

draw 是 virtual， display 不是。Circle 没有重写 draw，所以通过 Shape* 调用 draw 输出基类版本；Square 重写 draw，输出 square；Shape 对象本身输出 shape。

填空题 Q3

3. What are the output of the following code?

```

#include <iostream>
using namespace std;

template<class T>
class Item {
public:
    void print() { cout << "Item<T>" << endl; }
};

template<>
class Item<double> {
public:
    void print() { cout << "Item<double>" << endl; }
};

template<>
class Item<bool> {
public:
    void print() { cout << "Item<bool>" << endl; }
};

int main() {
    Item<int> item1;
    Item<float> item2;
    Item<double> item3;

    item1.print();
    item2.print();
    item3.print();

    return 0;
}

```

答案：

Item [1分]

Item [1分]

Item [1分]

解析：

模板全特化只匹配指定类型：Item<int> 和 Item<float> 使用主模板，Item<double> 使用 double 特化。原 HTML 答案显示为 Item，但按代码应理解为 Item<T>，Item<T>，Item<double>。

填空题 Q4

4. What are the output of the following code?

```
#include <iostream>
using namespace std;

class Counter {
public:
    Counter() {
        count++;
    }
    ~Counter() {
        count--;
    }
    static int getCount() {
        return count;
    }
private:
    static int count;
};

int Counter::count = 0;

void func() {
    Counter c1;
    cout << Counter::getCount() << endl;
}

int main() {
    func();
    cout << Counter::getCount() << endl;
    return 0;
}
```

答案：

1 [2分]

0 [2分]

解析：

func 内创建局部对象 c1，构造后 static count 为 1，函数结束析构后 count 回到 0，所以 main 中再次输出 0。

填空题 Q5

5. What are the output of the following code?

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void display() const {
        cout << "Base display" << endl;
    }
};

class Derived : public Base {
public:
    void display() const override {
        cout << "Derived display" << endl;
    }
};

int main() {
    Base* b = new Derived();
    Derived* d = dynamic_cast<Derived*>(b);
    if (d != nullptr) {
        d->display();
    } else {
        cout << "Conversion failed" << endl;
    }

    double pi = 3.14159;
    int i = static_cast<int>(pi);
    cout << i << endl;

    delete b;
    return 0;
}
```

答案:

Derived display [2分]

3 [2分]

解析：

Base 有虚函数，所以 `dynamic_cast<Derived*>(b)` 成功，调用 Derived 的 `display`；`static_cast<int>(3.14159)` 截断为 3。

填空题 Q6

6. What are the output of the following code?

```

#include <iostream>
using namespace std;

class Employee {
public:
    Employee() {
        cout << "Employee ctor called" << endl;
    }

    virtual ~Employee() {
        cout << "Employee dtor called" << endl;
    }

    virtual void show() const {
        cout << "Employee::show()" << endl;
    }
};

class Manager : public Employee {
public:
    Manager() {
        cout << "Manager ctor called" << endl;
    }

    ~Manager() {
        cout << "Manager dtor called" << endl;
    }

    void show() const override {
        cout << "Manager::show()" << endl;
    }
};

void display(Employee* b) {
    b->show();
}

int main() {
    Employee* b = new Manager();
    display(b);

    delete b;
}

```

```
    return 0;  
}
```

答案：

Employee ctor called [1分]

Manager ctor called [1分]

Manager::show() [1分]

Manager dtor called [1分]

Employee dtor called [1分]

解析：

new Manager() 先构造基类 Employee，再构造派生类 Manager； display(b) 通过虚函数调用 Manager::show； delete b 因为基类析构 virtual，先析构 Manager，再析构 Employee。

填空题 Q7

7. What are the output of the following code?

```

#include <iostream>
using namespace std;

namespace A {
    int value = 10;

    namespace B {
        int value = 20;

        int getValue() {
            return value - A::value;
        }
    }
}

namespace C {
    int value = 30;

    int getValue() {
        return value + A::B::getValue();
    }
}

int main() {
    cout << A::B::getValue() << endl;
    cout << C::getValue() << endl;
    cout << A::value + C::value << endl;
    return 0;
}

```

答案：

10 [1分]

40 [1分]

40 [1分]

解析：

A::B::getValue() 返回 20-10=10； C::getValue() 返回 30+10=40；最后 A::value + C::value 是 10+30=40。

填空题 Q8

8. What are the output of the following code?

```

#include <iostream>
#include <cstring>
using namespace std;

class DynamicArray {
public:
    DynamicArray(int size = 0) : size(size), data(new int[size]) {
        for (int i = 0; i < size; ++i) {
            data[i] = i+1;
        }
    }

    DynamicArray(const DynamicArray& other) : size(other.size), data(new int[other.size]) {
        for (int i = 0; i < other.size; ++i) {
            data[i] = other.data[i];
        }
    }

    DynamicArray& operator=(const DynamicArray& other) {
        if (this != &other) {
            delete[] data;
            size = other.size;
            data = new int[size];
            for (int i = 0; i < size; ++i) {
                data[i] = other.data[i];
            }
        }
        return *this;
    }

    DynamicArray operator+(const DynamicArray& other) {
        DynamicArray result(size + other.size);
        for (int i = 0; i < size; ++i) {
            result.data[i] = data[i];
        }
        for (int i = 0; i < other.size; ++i) {
            result.data[size + i] = other.data[i];
        }
        return result;
    }

    int& operator[](int index) {
        return data[index];
    }

```

```

}

friend ostream& operator<<(ostream& os, const DynamicArray& arr) {
    for (int i = 0; i < arr.size; ++i) {
        os << arr.data[i] << (i < arr.size - 1 ? ", " : "");
    }
    return os;
}

~DynamicArray() {
    delete[] data;
}

private:
    int size;
    int* data;
};

int main() {
    DynamicArray arr1(3);
    DynamicArray arr2(2);
    DynamicArray arr3;

    arr3 = arr1 + arr2;

    arr3[2] = 10;
    cout << arr3[0] << endl;
    cout << arr3[1] << endl;
    cout << arr3[2] << endl;
    cout << arr3[4] << endl;

    return 0;
}

```

答案：

1 [1分]

2 [1分]

10 [1分]

2 [1分]

解析：

arr1 + arr2 连接两个数组，初始为 [1,2,3,1,2]；赋给 arr3 后把 arr3[2] 改成 10，所以依次输

出 1、2、10、2。注意这里考深拷贝、赋值和 operator[] 。

程序填空题 Q1

1. Fill-in-the-Blank Question about Template Partial Specialization

```
#include <iostream>
using namespace std;

// Primary template
template <typename T>
class MyClass {
public:
    MyClass(T v): value(v) {}
    void Display() { cout << "Primary Template: " << value << endl; }

private:
    T value;
};

template <typename T>
class MyClass<填空处> // Blank 1: Declare class template for T*
{
public:
    MyClass(T* v): value(*v), pValue(v) {} // Blank 2: Initialize member variable
    void DisplayPtr() { cout << "Partial Specialization: " << pValue << endl; }
    void Display() { cout << "Partial Specialization: " << value << endl; }

private:
    T* pValue;
    T value;
};

int main() {
    MyClass<int> obj1(10);
    obj1.Display();

    int value = 20;
    MyClass<int*> obj2(填空处); // Blank 3: Initialize obj2 with partial specification
    obj2.填空处(); // Blank 4: Call the Display method for obj2

    return 0;
}
```

答案：

T* [2分]

*v [2分]

&value [2分]

Display [2分]

解析：

核心是模板偏特化： MyClass<T*> 专门处理指针类型；构造 MyClass<int*> obj2(&value) ，成员 value 初始化为 *v ，最后调用 obj2.Display() 输出偏特化版本。

程序填空题 Q2

2. Fill-in-the-Blank Question about Inheritance and Polymorphism

```

#include <iostream>
#include <string>
using namespace std;

class Shape {
public:
    Shape(const std::string& name): 填空处(name) {}
    virtual void calculateArea() {}
    void display() const {
        cout << mName << " area: " << mArea << endl;
    }

    填空处 ~Shape(){
        cout << "Destructor of Shape is called" << endl;
    }

protected:
    float mArea;

private:
    std::string mName;
};

class Circle : 填空处 Shape {
public:
    Circle(float r): Shape("Circle"), mRadius(r) {}
    void calculateArea() {
        mArea = 3.14 * mRadius * mRadius;
    }
    ~Circle() {
        cout << "Destructor of a subclass is called" << endl;
    }

private:
    float mRadius;
};

class Square : public Shape {
public:
    Square(float s): Shape("Square"), mSide(s) {}
    void calculateArea() {
        mArea = mSide * mSide;
    }

private:

```

```

    float mSide;
};

int main() {
    填空处 shapes[2];
    shapes[0] = new Circle(2.0);
    shapes[1] = new 填空处(4.0);

    for(int i = 0; i < 2; i++) {
        shapes[i]->calculateArea();
        shapes[i]->display();

        填空处 shapes[i];
    }

    return 0;
}

```

答案：

mName [2分]

virtual [2分]

public [2分]

Shape* [2分]

Square [2分]

delete [2分]

解析：

这些空都围绕多态类骨架：构造函数初始化 mName ，基类析构要 virtual ，Circle 要 public Shape ，数组类型是 Shape* shapes[2] ，创建 Square 后通过基类指针调用虚函数，最后 delete shapes[i] 。

主观题（20分） Q1

Online Support Chat System

We aim to design an Online Support Chat System that routes customer queries to appropriate support agents based on their expertise. The system follows the Chain of Responsibility Design Pattern, allowing queries to be processed step by step through a chain of handlers.

There are 6 classes in total in the system:

SupportQuery: A class representing a customer query, with the following attribute:

- type: The type of query (e.g., "Billing", "Technical", "Customer Service").

Handler: An abstract class representing a generic handler in the chain. It declares:

- SetNext(Handler handler)*: Links the current handler to the next handler in the chain.
- Handle(SupportQuery& query): Processes or forwards the query.

BaseHandler: A class derived from Handler implementing default chaining behavior. It forwards queries to the next handler if the current handler cannot process them.

BillingHandler: A concrete handler that processes queries of type "Billing". If the query is not "Billing", it will forward the query to the next handler in the chain.

TechnicalHandler: A concrete handler that processes queries of type "Technical". If the query is not "Technical", it will forward the query to the next handler in the chain.

CustomerServiceHandler: A concrete handler that processes queries of type "Customer Service". If the query is not "Customer Service", it will forward the query to the next handler in the chain.

The implementation for BaseHandler and BillingHandler are as follows:

```

// Implements chaining logic class
// BaseHandler implements chaining logic
class BaseHandler : public Handler {
private:
    Handler* nextHandler = nullptr;
public:
    Handler* SetNext(Handler* handler) override {
        nextHandler = handler;
        return handler;
    }
    string Handle(SupportQuery& query) override {
        if (nextHandler) {
            return nextHandler->Handle(query);
        }
        return {};
    }
};

// Concrete Handlers
class BillingHandler : public BaseHandler {
public:
    string Handle(SupportQuery& query) override {
        if (query.getType() == "Billing") {
            return "BillingHandler: Handling Billing query.\n";
        } else {
            return BaseHandler::Handle(query);
        }
    }
};

```

Your task is to implement SupportQuery, Handler, TechnicalHandler, and CustomerServiceHandler classes, in order to make sure that the following code works properly.

```

#include <iostream>
#include <string>
#include <vector>
using namespace std;

// Client code to test the chain
void ClientCode(Handler& handler, vector<SupportQuery>& queries) {
    for (auto& query : queries) {
        cout << "Processing query (Type: " << query.getType() << ")\n";
        const string result = handler.Handle(query);
        if (!result.empty()) {
            cout << " " << result;
        } else {
            cout << " " << query.getType() << " query was left unhandled.\n";
        }
    }
}

int main() {
    // Create handlers
    BillingHandler* billing = new BillingHandler;
    TechnicalHandler* technical = new TechnicalHandler;
    CustomerServiceHandler* customerService = new CustomerServiceHandler;

    // Build the chain
    billing->SetNext(technical)->SetNext(customerService);

    // Test queries
    vector<SupportQuery> queries = {
        SupportQuery("Billing"),
        SupportQuery("Technical"),
        SupportQuery("Customer Service")
    };
    cout << "Chain: Billing > Technical > Customer Service" << endl;
    ClientCode(*billing, queries);
    cout << endl;
    cout << "Subchain: Technical > Customer Service" << endl;
    ClientCode(*technical, queries);

    // Clean up
    delete billing;
    delete technical;
    delete customerService;
}

```

```
    return 0;
}
```

The output of the program will be:

```
Chain: Billing > Technical > Customer Service
Processing query (Type: Billing)
    BillingHandler: Handling Billing query.
Processing query (Type: Technical)
    TechnicalHandler: Handling Technical query.
Processing query (Type: Customer Service)
    CustomerServiceHandler: Handling Customer Service query.

Subchain: Technical > Customer Service
Processing query (Type: Billing)
    Billing query was left unhandled.
Processing query (Type: Technical)
    TechnicalHandler: Handling Technical query.
Processing query (Type: Customer Service)
    CustomerServiceHandler: Handling Customer Service query.
```

解析：

这是责任链模式。SupportQuery 只需保存 type 并提供 getType()；Handler 声明 SetNext 和 Handle 两个纯虚接口；Technical 和 CustomerService 与 BillingHandler 同套路：能处理就返回对应字符串，不能处理就调用 BaseHandler::Handle(query) 传给下一个处理者。

2023-2024 春夏原题

单选题

单选题 Q1

1. Copy constructor is a constructor which _____
- A. Creates an object by copying values from another object of another class
 - B. Creates an object by copying values from any other object of same class
 - C. Creates an object by initializing it with another previously created object of same class
 - D. Creates an object by copying values from first object created for that class

解析：

选 C。拷贝构造是用同一类已经存在的对象来初始化一个新对象。

单选题 Q2

2. In multi-level inheritance(all public), the public members of parent/superclass will _____
- A. Will continue to get inherited subsequently
 - B. Will not be available to be called outside class
 - C. Will not be inherited after one subclass inheritance
 - D. Will not be able to allocated with any memory space

解析：

选 A。public 多级继承中，基类 public 成员会继续以 public 身份被后续派生类继承。

单选题 Q3

3. Which code below fails compilation?

A.

```
struct U {};  
struct V : public U {};  
struct W : public U {};  
int main()  
{  
    U * p = new V;  
    W * q = static_cast<W*>(p);  
    return q == nullptr;  
}
```

B.

```
struct U { virtual void foo() {} };  
struct V : public U {};  
struct W {};  
int main()  
{  
    U * p = new V;  
    W * q = dynamic_cast<W*>(p);  
    return q == nullptr;  
}
```

C.

```
struct U { virtual void foo() {} };  
struct V : public U {};  
struct W : public U {};  
int main()  
{
```

```

U * p = new V;
W * q = dynamic_cast<W*>(p);
return q == nullptr;
}
D.
struct U {};
struct V : public U {};
struct W {};
int main()
{
U * p = new V;
W * q = static_cast<W*>(p);
return q == nullptr;
}

```

解析：

考强制类型转换。dynamic_cast 到无继承关系类型会编译失败；static_cast 也要求类型之间存在允许的转换关系。本题原卷也标注第 3 题答案不确定，复习时重点掌握：dynamic_cast 需要多态基类且只能在合理继承层次中安全转换。

单选题 Q4

4. If programmer have defined parameterized constructor only, then _____
- A. Default constructor will not be created by the compiler implicitly
 - B. Default constructor will not be created but called at runtime
 - C. Default constructor will be created by the compiler implicitly
 - D. Compile time error

解析：

选 A。只自定义有参构造后，编译器不再隐式生成无参默认构造。

单选题 Q5

5. Which of the following operator cannot be used to overload when that function is declared as friend function?
- A. ||
 - B. ==
 - C. -=
 - D. []

解析：

选 D。operator[] 必须是成员函数，不能写成普通 friend 非成员函数。

单选题 Q6

6. If a class have default constructor defined in private access, and one parameter constructor in protected mode, how will it be possible to create instance of object?

- A. Directly create the object in the subclass
- B. Define a constructor in public access with different signature
- C. Directly create the object in main() function
- D. Not possible

-- 马二马纯上 --

解析：

选 B。默认构造 private、有参构造 protected 都不能从 main 直接调用；但可以在 public 区域再定义一个不同签名构造函数，由它间接调用可访问的构造。

单选题 Q7

7. Which programming paradigm below is not well supported in C++?

- A. Object-oriented programming
- B. Procedural programming
- C. Declarative programming
- D. Generic programming

解析：

选 C。C++ 强支持过程式、面向对象、泛型；声明式不是它的主要范式。

单选题 Q8

8. Which statement below is not a requirement for an object-oriented programming language?

- A. Objects have an associated type [class].
- B. Types [classes] may inherit attributes from supertypes [superclasses].
- C. It uses algorithms as its fundamental logical building blocks.
- D. It supports objects that are data abstractions with an interface of named operations and a hidden local state.

解析：

选 C。OOP 的核心是对象、类、封装、接口、继承/多态，不是“算法作为基本逻辑构件”。

单选题 Q9

9. Which among the following is multiple inheritance?

A.

```
class student { };  
class stream { };  
class topper { };
```

B.

```
class student {  
public:  
int marks;  
} s;  
class stream {  
int total;  
};  
class topper : public student, public stream { };
```

C.

```
class student {  
int marks;  
};  
class stream { };  
class topper : public student { };
```

D.

```
class student {  
int marks;  
};  
class stream : public student { };
```

解析：

选 B。class topper : public student, public stream 同时继承两个基类，是多继承。

单选题 Q10

10. How many times is the destructor of class A called during execution?

```
#include  
using namespace std;  
class A {  
public:  
A() { cout << "A()" << endl; }  
~A() { cout << "~A()" << endl; }
```

```
};
void foo(A a) {
    A arr[5];
    A *p = new A[3];
    throw p;
}
int main()
{
    try {
        A a;
        foo(a);
    } catch (A *p) {
        delete[] p;
        cout << "caught" << endl;
    }
}
```

A. 5
B. 7
C. 10
D. 3

解析：

foo(a) 按值传参会拷贝一个参数对象；函数内构造 arr[5] 和 new A[3] 。抛异常后局部数组和参数会析构，catch 中 delete[] p 析构 3 个动态对象，main 里的 a 最后析构；合计 10 次析构。

结果填空题

结果填空题 Q1

1.

```
#include <iostream.h>
#include <string.h>
#include <cstdlib.h>
using namespace std;
void func(int a, int b)
{
    if (b < 1) {
        throw b;
    }
}
```

```

else {
cout << "Product of " << a << " and " << b << " is: " << ab << endl;
}
}
int main()
{
try
{
try
{
func(5,-1);
}
catch (int b)
{
if (b==0)
throw "value of b is zero\n";
else
throw "value of b is less than zero\n";
}
}
catch (const char e)
{
cout << e;
}
}

```

Line 1: _____ (3分)

-- CC98 马二马纯上 --

解析：

func(5,-1) 中 b<1, 抛出 int -1; 内层 catch 判断 b 不是 0, 于是抛出字符串 value of b is less than zero\n; 外层 catch 捕获 const char* 并输出它。

结果填空题 Q2

2. The output of the code below is:

```

#include
using namespace std;
class AnyError
{

```

```

public:
AnyError() { }
};
class RangeError :public AnyError
{
public:
RangeError(int ID) { m_ID = ID; }
void print() { cout << "ID:" << m_ID << endl; }
protected:
int m_ID;
};
void FuncA(int ID)
{
if (ID > 8)
throw RangeError(ID);
cout << "FuncA" << endl;
}
void FuncB()
{
try
{
FuncA(4);
FuncA(10);
}
catch (RangeError& re)
{
cout << "FuncB" << endl;
re.print();
throw AnyError();
}
}
int main()
{
try {
FuncB();
}
catch (...)
{

```

```
cout << "Main" << endl;
}
return 0;
}
```

The first line is: _____ (1分)

The second line is: _____ (1分)

The third line is: _____ (1分)

The fourth line is: _____ (1分)

3.

```
#include
using namespace std;
template
class A {
public:
A() {}
void print() const
{
cout << "A" << endl;
}
};
template <>
class A {
double a1, a2, a3;
public:
A()
{}
void print() const
{
cout << "A" << endl;
return;
}
};
template <>
class A {
double a1, a2, a3;
public:
A()
```

```

{}
void print() const
{
cout << "A" << endl;
return;
}
};
int main()
{
A t1;
A t2;
A t3;
t1.print();
t2.print();
t3.print();
}

```

The output of this program should be:

Line 1: _____ (1分)

Line 2: _____ (1分)

Line 3: _____ (1分)

解析：

前半题： FuncA(4) 输出 FuncA； FuncA(10) 抛 RangeError，被 FuncB 捕获，输出 FuncB 和 ID:10，再抛 AnyError，被 main 的 catch(..) 捕获输出 Main。后半题模板特化： double 用 A<double> ， char 用主模板 A<T> ， int 用 A<int> 。

结果填空题 Q4

4. What is the output of the following code?

```

#include
#include
using namespace std;
class WeirdString
{
public:
WeirdString() = default;
WeirdString& operator++() {
s += "Hey";
return *this;
}
}

```

```

}
WeirdString operator++(int) {
WeirdString old = *this;
s += "Ho";
return old;
}
WeirdString& operator+=(const WeirdString& rhs) {
s += ("Ha" + rhs.str());
return *this;
}
string str() const { return s; }
private:
string s;
};
-- CC98 MaErMaChunShang --
int main()
{
WeirdString w;
WeirdString t = w++;
++(w += (t++));
cout << w.str() << endl;
}

```

Line 1: _____ (3分)

5.

Content of header file h1.h

```

// h1.h
#include <iostream.h>
using namespace std;
namespace A {
int func(int a) {
cout << "using namespace A";
return 2a;
}
}

```

Content of header file h2.h

```

// h2.h
#include <iostream.h>

```

```
using namespace std;
namespace B {
float func(float a) {
cout << "using namespace B";
return 2a;
}
}
```

Content of program.cpp

```
#include <iostream.h>
#include <string.h>
#include "h1.h"
#include "h2.h"
using namespace std;
using namespace A;
using namespace B;
int main(int argc, char const argv[]) {
/ code */
int a = 10;
float b = 10.0;
cout << func(a) << endl;
cout << func(b);
return 0;
}
```

Line 1: _____ (2分)

Line 2: _____ (2分)

解析：

前半题：后置 `w++` 返回旧值但让 `w` 加 `Ho`；`t++` 返回旧 `t`；`w += old_t` 追加 `Ha` 和旧 `t` 的字符串；前置 `++` 再追加 `Hey`，所以输出 `HoHaHey`。后半题 `namespace`：`int` 参数选 `A::func`，`float` 参数选 `B::func`，输出函数内提示文字再输出返回值。

结果填空题 Q6

6. What will be the output of the following code?

```
#define MaErMaChunShang_CC98
#include
using namespace std;
class C1
{
```

```

public:
C1() {
cout << "C1()";
}
C1(const C1& a) {
cout << "ParseError: KaTeX parse error: Expected 'EOF', got '&' at position 12: C1(const
C1&);";
}
virtual ~C1() {
cout << " C1()";
}
};
class C2 : public C1
{
public:
C2() {
cout << "C2()";
}
~C2() {
cout << " C2()";
}
};
int main()
{
C2* pC2 = new C2();
cout << endl;
{
C1 a = pC2;
cout << endl;
}
C1 pC1 = pC2;
delete pC1;
cout << endl;
}

```

The output is:

Line 1: _____ (1分)

Line 2: _____ (1分)

Line 3: _____ (1分)

解析：

第一行 new C2：先构造 C1 再构造 C2；第二行 C1 a = *pC2 发生对象切片，调用 C1 拷贝构造，块结束析构这个 C1；第三行通过 C1* delete C2，因为 C1 析构 virtual，所以先 C2 析构再 C1 析构。

程序填空题

程序填空题 Q1

1. Array is a template that implements a dynamically expandable array. next is used to link to the next block of the array. Fill in the blanks to complete the code.

```
#include
using namespace std;
template
class Array {
public:
Array() {
data = new T[BLK_SIZE];
next = _____(1分);
}
~Array() {
delete [] data;
delete next;
}
T& operator[](int i);
void iterate(void (*f)(T&));
private:
_____(2分) *data; // data of type T
static const int BLK_SIZE=32; // fixed block size
_____(1分) *next; // the next array block
};
template
T& _____(1分) operator[](int i) {
if (i < BLK_SIZE) {
return _____(2分);
} else {
if (next == NULL) {
next = new _____(2分);
}
return (*next)[i-BLK_SIZE];
}
```

```

}
}
template
void _____(2分) iterate(void (*f)(T&)) {
for (int i = 0; i < BLK_SIZE; i++) {
f(data[i]);
}
if (next != NULL) {
next-> _____(2分);
}
}
int main()
{
Array _____(2分) a;
int size = 100;
cin >> size;
for (int i = 0; i < size; i++) {
a[i] = i;
}
a.iterate([](int &x) { cout << x << endl; });
}
-- CC98 马二马纯上 --

```

解析：

这是分块动态数组。空位依次围绕 `next = NULL` 、 `T *data` 、 `Array<T> *next` 、 类外定义 `Array<T>::operator[]` 、 返回 `data[i]` 、 需要时 `new Array<T>` 、 `Array<T>::iterate` 、 递归 `iterate(f)` 、 以及 `Array<int> a` 。

程序填空题 Q2

2. The function `inner_product` computes the inner product (i.e., sum of products) between the elements in the range `[first1, last1)` and those in the same size range beginning at `first2`.

Please fill in the blanks of the following code to finish the implementation.

The code to complete:

```

#include
#include
#include
_____(1分) <class InputIt1, class InputIt2, class T, class BinaryOp1, class
BinaryOp2>

```

```

T inner_product(InputIt1 first1, InputIt1 last1, InputIt2 first2, T init,
_____(3分) op1, _____(3分) op2)
{
while (first1 != last1)
{
init = _____(3分) (init, op2( _____(3分) ));
++first1;
_____(2分) ;
}
return init;
}

int main()
{
std::vector a{0, 1, 2, 3, 4};
std::vector b{5, 4, 2, 3, 1};
int r1 = inner_product(a.begin(), a.end(), b.begin(), 0, std::plus<>(),
std::multiplies<>());
std::cout << "Inner product of a and b: " << r1 << '\n';
int r2 = inner_product(a.begin(), a.end(), b.begin(), 0, std::plus<>(),
std::equal_to<>());
std::cout << "Number of pairwise matches between a and b: " << r2 << '\n';
}

```

Required output:

Inner product of a and b: 21

Number of pairwise matches between a and b: 2

解析：

这是仿 STL 的 inner_product。模板参数包括两个迭代器、初值类型和两个二元操作；循环中用 op2(*first1, *first2) 先组合一对元素，再用 op1(init, ...) 累加，并同时递增 first1 和 first2。

主观题

主观题 Q?

The class hierarchy in this program design task is as follows:

Note that the shapes are contained in the CShapeDocument, and they are read from the file using its member function parse(ifstream &if). The text stored in the file to describe the shapes in the document are as follows:

Rectangle left-bottom 1.0 1.0 W&H 10.0 20.0

Circle center 10.0 10 radius 5.0

Ellipse center 9.0 9.0 radius 10.0 30.0

The first string is the type of the shape and the rest texts describe its attributes. Specifically, left-bottom means 1 and 1 are the 2D coordinate of the rectangle, and W&H means 10 and 20 are the width and height of the rectangle. The descriptions of the circle and ellipse are similar.

Suppose the following texts are stored in the D:\shapes.txt:

Rectangle left-bottom 20.0 50.0 W&H 20.0 40.0

Circle center 8.0 8.0 radius 7.0

Ellipse center 5.0 5.0 radius 10.0 2.0

Please read the implementation of CShapeDocument and main function and implement the required shape classes, such as Rectangle, Ellipse and Circle such that the shapes are analyzed from the file D:\shapes.txt and stored in an object of the class CShapeDocument.

```
#include
#include
#include
using namespace std;
class shape
{
public:
virtual ~shape() {}
virtual void draw() = 0;
virtual bool parseattribute(ifstream &ifstream) = 0;
};
class Rectangle : public shape
{
...
};
class Circle : public shape
{
...
};
class Ellipse : public shape
{
...
};
== CC98 马二马纯上 ==
class CShapeDocument
```

```

{
public:
CShapeDocument() {}
~CShapeDocument();
public:
void draw();
void parse(ifstream& ifs);
private:
std::vector<shape*> m_vShape;
};
CShapeDocument::~~CShapeDocument()
{
std::vector<shape*>::iterator itShape;
for (itShape = m_vShape.begin(); itShape != m_vShape.end(); itShape++)
delete* itShape;
}
void CShapeDocument::draw()
{
std::vector<shape*>::iterator itShape;
for (itShape = m_vShape.begin(); itShape != m_vShape.end(); itShape++)
(itShape)->draw();
}
template
void parseattribute(ifstream& ifs, std::vector<shape> &vShape)
{
shape* pX = new X();
if (!pX->parseattribute(ifs))
{
delete pX;
return;
}
vShape.push_back(pX);
}
void CShapeDocument::parse(ifstream &ifs)
{
std::string str;
while(1)
{

```

```

str = "";
ifs >> str;
if (str == "Rectangle")
parseattribute(ifs, m_vShape);
else if (str == "Circle")
parseattribute(ifs, m_vShape);
else if (str == "Ellipse")
parseattribute(ifs, m_vShape);
else
break;
}
}
int main()
{
std::ifstream ifs("D:\\shape.txt");
CShapeDocument shapedoc;
shapedoc.parse(ifs);
shapedoc.draw();
}

```

And the output of the program is as follows:

Rectangle 20 50 W&H 20 40

Circle 8 8 radius 7

Ellipse 5 5 radius 10 2

解析：

Shape 大题的套路是抽象基类 + 派生类解析属性 + 多态 draw。Rectangle/Circle/Ellipse 各自保存坐标和尺寸， parseattribute(ifstream&) 按固定单词和数字读入， draw() 按样例格式输出；

CShapeDocument 已负责根据类型字符串 new 对应派生类并保存到 vector<shape*> 。

单选题答案

CADAD BCCBC

注：单选第3题答案不能确定，待考

马二马纯上 浙江大学校内论坛

请勿用于商业用途，转载请与我取得联系

2019-2020 春夏第一部分：程序填空题（可抽取文本）

5-1

5-1

```
#include
作者 许威威
#include
using namespace std; 单位 浙江大学
时间限制 400 ms
内存限制 64 MB
friend class Linklist;
Node *m_pNext;
public:
Node(){ m_pNext = NULL; }
virtual ~Node() {}
void AppendNode(Node &n){n.m_pNext = m_pNext; (1 分);}
virtual void Print()const = 0;
},
class IntNode : public Node{
int m_i;
public:
IntNode(int i){m_i = i;}
virtual void Print()const{cout << m_i << endl;}
},
class StrNode : public Node
{
char *m_s;
public:
StrNode(char *s){ m_s = new char[ (1分)]; strcpy(m_s,s);}
~StrNode (){ (1分)},
virtual void Print()const{cout << m_s << endl;}
};
class Linklist
{
Node *m_pHead;
Node *m_pEnd;
```

```

public:
linklist (Node &n){ m_pHead = m_pEnd = &n;}
~linklist(){
Node *p = m_pHead;
while (p){ (1分) ; delete p ; (1分) ;}
}
void AppendNode(Node &n){
m_pEnd->AppendNode(n);
m_pEnd = (1 分) ;
}
void PrintList() (1分) {
Node *p = m_pHead;
while (p){p->Print(); (1 分) ; }
},
int main()
{
char word[80] ;
cin >> word;
LinkedList llist( (1分) StrNode(word));
int i;
cin >> i;
llist.AppendNode( (1分) IntNode(i));
llist. PrintList();
}

```

解析：

这是多态链表程序填空。关键点：Node 是抽象基类，IntNode/StrNode 实现 Print；StrNode 要深拷贝字符串并在析构释放；LinkedList 析构要沿 next 删除节点；AppendNode 要维护尾指针；main 里动态创建节点，否则局部临时对象会悬空。

5-1

5-1

解析：

这是多态链表程序填空。关键点：Node 是抽象基类，IntNode/StrNode 实现 Print；StrNode 要深拷贝字符串并在析构释放；LinkedList 析构要沿 next 删除节点；AppendNode 要维护尾指针；main 里动态创建节点，否则局部临时对象会悬空。

5-2

5-2 要求实现一个表示分数的Fraction类中的部分函数 注意 计结果不要求约分。

， ，

作古 许威威

class Fraction{

单位 浙江大字

double numerator;

时间限制 400 ms

double denominator;

public: 内存限制 64 MB

Fraction (double numerator•0. 0, double denominator•.0 0){

(1分),

(1分)

}

Fraction ((1 分) fl){

numerator •f l. numerator;

(1分) •fl. denominator;

}

Fraction (1 分) + (const Fraction &fl);

(1分) operator— ((1 分)) {

return Fraction((numerator,.fl.d enominator • numeratorf2. denominator (1 分)),

(denominatorfl.denominator));

}

(1 分) void print(Fraction f);

},

Fraction Fraction: :operator+(const Fraction &fl){

return (1 分) ;

}

void print(Fraction f)

{

cout << "numerator " << f, numerator << endl;

cout << "denominator " << f.denominator << endl;

}

解析:

Fraction 题主要考构造函数默认参数、拷贝构造、运算符重载和 friend/普通函数访问私有成员。加法分子为 $n1*d2 + n2*d1$, 分母为 $d1*d2$; 减法同理把加号换成减号。题目说明不要求约分。

5-2

5-2

解析：

Fraction 题主要考构造函数默认参数、拷贝构造、运算符重载和 friend/普通函数访问私有成员。加法分子为 $n1*d2 + n2*d1$ ，分母为 $d1*d2$ ；减法同理把加号换成减号。题目说明不要求约分。

2018-2019 原题（OCR 摘录）

说明：该 PDF 是图片式页面，以下为 OCR 识别文本，代码符号可能需要对照原 PDF 校正。

2018-2019 page 01

2018-2019 OOP ##*#i

/x True-or-False 10

A. Multiple-Choice - 1 10

A Fill-in-Blank 9

@ Fill-in-Blank - P 3

1-1 catch (type p) acts very much like a parameter in a function. Once the exception is thrown value from this parameter in the body of a catch block.. (25)

T

> F

1-1 Accepted (2 point(s))

1-2 Inserter («<) can be used to output all kinds of primitive types, including the poi

• F

1-2 Accepted (2 points))

1-3 The reason inline functions are introduced into the C++ is to reduce the complexity of code. (25)

• F

1-3 Accepted (2 points))

1-4 In C++, only existing operators can be overloaded. (25)

T

• F

1-4 Accepted (2 points))

1-5 If you are not interested in the contents of an exception object, the catch block p

• F

1-5 Wrong Answer (0 points))

1-6 It is possible to access any item in a vector directly via its index. (25)

• T

• F

1-6 Accepted (2 point(s))

1-7 Functions with the same name can be identified via namespaces. (25)

T

F

1-7 Accepted (2 point(s))

1-8 To make functions overloaded, the parameter list of the functions have to be different

T

• F

1-8 Wrong Answer (0 points))

1-9 Constructors are able to be declared as virtual. (25)

T

• F

1-9 Accepted (2 points))

1-10 Manipulators are objects to be inserted or extracted into/from streams. (25)

• T

) F

1-10 Accepted (2 points))

Author: 3K152

Organization: Æzz®B

Author: sat#

Organization: #fIX·

Author: flE

Organization: It

Author: S6E

Organization: Z#A·

Author: 5K8-

Organization: FEEt

Author: ala

Organization: *FIX*

Author: 6alW

Organization: HIIt÷

Author: fat#

Organization: fIX÷

Author: fat#

Organization: HIIt·

Author: fat#

Organization: 9X9

解析：

本页是判断题。考点依次是：catch 参数、流插入器、inline 目的、只能重载已有运算符、catch 参数名可省略、vector 下标访问、namespace 区分同名函数、函数重载参数表、构造函数不能 virtual、manipulator 属于流操作对象。OCR 中选项圆点不可靠，复习时按这些规则判断。

2018-2019 #i

1½ True-or-False 10

A. Multiple-Choice – 1 10

@ Fill-in-Blank 9

@ Fill-in-Blank – P 3

2-1 Given code below:

```
vector<int> v;  
for (int i=0; i<4; i++ ) {  
v.push_back(i+1);
```

The output should be: (25)

A. 1

B. 2

C. 3

• D. 4

2-1 Accepted (2 point(s))

2-2 About virtual function, which statement below is correct? (25))

A. Virtual function is a static member function

B. Virtual function is not a member function

C. Once defined as virtual, it is still virtual in derived class without virtual keyword

D. Virtual function can not be overloaded.

2-2 Accepted (2 points))

2-3 It is better to choose when the function is not complicated and is to be called frequently

A. overloaded function

• B. inline function

C. recursive function

D. embedded function

2-3 Accepted (2 points))

2-4 Suppose that statement3 throws an exception of type Exception3 in the following statement block

```
statement; statement; statement3; }  
catch (Exception1 ex1) {}/  
catch (Exception2 ex2) {}  
catch (Exception3 ex3) { statement4; throw; }  
statement5;
```

Which statements are executed after statement3 is executed?

• A. statement2

B. statement3

C. statement4

D. statement5

2-4 Accepted (2 points))

2-5 What is wrong in the following code?

vector v; v[0] = 2.5; (25)

- A. The program has a compile error because there are no elements in the vector.
- B. The program has a compile error because you cannot assign a double value to v[0].
- C. The program has a runtime error because there are no elements in the vector.
- D. The program has a runtime error because you cannot assign a double value to v[0].

2-5 Wrong Answer (0 points))

2-6 Given:

```
template <class T>
void max (T a, Tb, r 8c)
c = atb;
```

Which code fragement below is correct? (25)

- A. (int x,y; char z;max(x,y,z):
- B. double x,y;double z;max (x,y, z) ;
- C. (int x,y;float z;max (x,y,z) ;
- D. float x,y; double z;max (x,y, z) ;

2-6 Accepted (2 point(s))

2-7 About const data member, which statement below is correct? (25))

- A. const member can be defined without any initialization, and can not be modified.
- B. const member has to be initialized, and can not be modified.
- C. const member can be defined without any initialization, and can be modified later.
- D. const) member has to be initialized, and can be modified later.

2-7 Accepted (2 point(s))

2-8 Which operator below can not be overloaded? (25)

- A. 0.0
- B.I]
- C.::
- D. <<

2-8 Accepted (2 point(s))

2-9 Which one below can NOT be overloaded? (25)

- A. member function
- B. free function (global function)
- C. destructor
- D. constructor

2-9 Accepted (2 point(s))

2-10 About delete operator, which statement below is NOT correct? (25))

- A. Only pointers as the result of a new opertion can be used to be delete d.
- B. Destructor will be called automatically during the delete operation.
- C. It is safe to delete the same pointer multiple times.
- D. There's only one pair of [1 followed to delete a multi-dimension array.

2-10 Accepted (2 point(s))

Author: s

Organization: #It#

Organization: #It#

Author: fl
Organization: WIT
Author: 36y
Organization: #ŝt##
Author: 35y
Organization: #≥#**
Author: 2a18
Organization: HIS
Author: aa18
Organization: *FI÷
Author:
Organization: fIt#
Author:
Organization: #I÷
Author: a18
Organization: FI

解析:

本页是单选题。重点: vector push_back 后 size; virtual 在派生类中保持虚; 短小频繁调用函数适合 inline; 异常 catch 后重新 throw 的控制流; 空 vector 直接 v[0] 是运行时越界; 模板参数类型必须一致; const 数据成员必须初始化; :: 不能重载; 构造/析构不能作为普通函数重载; 重复 delete 同一指针不安全。

2018–2019 OP #*i

× True-or-False 10

A. Multiple-Choice – 1 10

4–1 The output of the code below is:

```
#includeriostream>
using namespace std;
class MyClass {
public:
Myclass(int x): val(x) {}
void Print() const {cout << 1 << val;}
void Print() {cout << 2 << val;}
private.
int val;
};
int main() {
const Myclass obj1(10);
Myclass obj2(20);
obji. Print ();
obj2.Print();
return 0;
```

A Fill-in-Blank 9

@ Fill-in-Blank – P 3

Author: ta

Organization: #fIt.

110220

(355)

4–1 Accepted (3 point(s))

4–2 The output of the code below is:

```
#include<iostream>
using namespace std;
class AA {
public:
AA () { cout << 1; }
~AA () { cout << 2; }
};
class BB: public AA {
AA aa;
public:
BB() { cout << 3; }
~BB() { cout << 4; }
int main () {
```

```

BB bb;
return 0;
113422
4-2 Accepted (3 point(s))
4-3 The output of the code below is:
#include <iostream>
using namespace std;
class A {
public:
A() { cout << 1; }
} a;
int main ()
cout << 2;
A a;
return 0;
Author: s
Organization: #fIt·
Author: Fl
Organization: #FIA÷
121
4-3 Accepted (3 point(s))
4-4 write the output of the code below.
#include<iostream>
using namespace std;
• Author: hulanqing
Organization: #It#

```

解析：

本页开始是代码输出题。Q4-1 考 const 对象调用 const 成员函数、普通对象优先调用非 const 版本；Q4-2 考继承和成员对象的构造析构顺序；Q4-3 考全局对象先于 main 构造、局部对象离开作用域析构。

```
class INCREMENT
public:
INCREMENT ( int V = 0, int i = 1);
void addIncrement ()
V += increment;
void print() const;
int get () const
private:
int v;
const int increment;
INCREMENT:: INCREMENT ( int V, int i ) : v(v), increment( 1 )
void INCREMENT: :print () const
cout << v << endl;
int main ()
INCREMENT value ( 1, 2);
value.print();
for ( int j = 1; j <= 2; j++ )
value.addIncrement ();
value.print();
return 0;
```

One for each line:

line 1:1

line 3:5

4-4 Accepted (3 point(s))

4-5 write the output of the code below.

```
#include<iostream>
using namespace std;
class TEST
int num;
public:
TEST ( int num=0);
void increment () ;
~TEST( );
TEST: : TEST(int num) : num(num)
cout << num << endl;
}
void TEST: : increment ()
num++;
}
TEST::~~TEST ()
cout << num << endl;
```

```
int main()
TEST array[2];
array[0]. increment);
array[1]. increment);
return 0;
(155) line 2:3
(155)
(155)
• Author: hulanqing
Organization: #FI
```

解析：

本页延续代码输出。INCREMENT 题考初始化列表和 const 成员只能初始化不能赋值；TEST 数组题考数组元素按下标顺序构造、逆序析构，以及对象成员值变化后析构输出。

One for each line:

line 1:0

line 2:0

line 3:1

line 4:1

(155)

(155)

(153)

(155)

4-5 Accepted (4 points))

4-6 The output of the code below is:

```
#include <iostream>
using namespace std;
class MyClass {
public:
    MyClass () {
        ++count;
    }
    ~MyClass () {
        --count;
    }
    static int getcount() {
        return count;
    }
private:
    static int count;
};
int MyClass::count = 0;
int main () {
    MyClass obj;
    cout << obj.getcount() ;
    MyClass obj2;
    cout << MyClass::getcount();
    cout << obj2.getcount ();
    return 0;
}
```

122

(35))

4-6 Accepted (3 point(s))

4-7 write the output of the code below.

```
#include<iostream>
using namespace std;
enum NOTE { middlec, Csharp, Cflat };
class Instrument {
```

```

virtual void play (NOTE) const = 0;
virtual char* what() const = 0;
virtual void adjust (int) = 0;
class Wind : public Instrument {
public:
void play(NOTE) const {
cout << 1 << endl;
}
char* what() const { return "Wind"; }
void adjust (int) {
class Percussion : public Instrument {
public:
void play (NOTE) const {
cout << 2 << endl;
char* what () const { return "Percussion"; }
void adjust (int) {
};
class Stringed : public Instrument {
public:
void play (NOTE) const {
cout << 3 << endl;
char* what() const { return "stringed"; }
Author: all
Organization: #FIIt·
• Author: hulanqing
Organization: FIx*

```

解析：

本页考 static 成员计数和抽象基类多态。static count 属于类，所有对象共享；Instrument 题用基类引用调用纯虚接口的派生实现，输出取决于实际对象类型。

```
vold adjust (Int) i
class Brass : public Wind {
public:
void play(NOTE) const {
cout << 11 << endl;
char* what () const { return "Brass"; }
};
class Woodwind : public Wind {
public:
void play(NOTE) const {
cout << 12 << endl;
char* what() const { return "Woodwind"; }
void tune (Instrument& i) {
i. play(middlec);
void f(Instrument& i) { i. adjust(1); }
int main() {
Wind flute;
Percussion drum;
stringed violin;
Brass flugelhorn;
Woodwind recorder;
tune (flute);
tune (drum);
tune(violin);
tune (flugelhorn);
tune (recorder);
f (flugelhorn);
return 0;
One for each line:
line 1:1
line 2:2
line 3:3
line 4:11
line 5:12
(155)
(155)
(155)
(155)
(15)
```

4-7 Accepted (5 point(s))

4-8 write the output of the code below.

```

#include<iostream>
#include<string>
using namespace std;
class Pet {
public:
virtual string speak() const { return "pet!"; }
};
class Dog : public Pet (
public:
string speak() const { return "dog!"; }
};
int main ()
Dog ralph;
Pet* p1 = &ralph;
Pet& pz = ralph;
Pet p3;
cout << p1->speak() <<endl;
cout << p2. speak() << endl;
cout << p3. speak() << endl;
return 0;
dog!
doe!
(15)
(15))
• Author: hulanqing
Organization: #FI÷

```

解析：

本页延续多态输出。tune(Instrument&) 通过引用保留动态类型，所以 Wind/Percussion/Stringed/Brass/Woodwind 分别调用自己的 play；Pet/Dog 题说明指针和引用调用 virtual 函数会动态绑定，普通 Pet 对象则输出基类版本。

```
pet!  
(155)  
4-8 Accepted (3 point(s))  
4-9 The output of the code below is:  
#include <iostream>  
using namespace std;  
class A {  
    int i;  
public:  
    A() : i (0) {}  
    ~A() { cout << get (); }  
    void set(int i) { this->i = i; }  
    int get() { return i; }  
};  
int main()  
{  
    A* p = new A[2];  
    delete p;  
    return 0;  
}  
0 (355)  
4-9 Accepted (3 points))  
Author: fa  
Organization: #fIt÷
```

解析：

本页主要考 new[] 与 delete[] 。题中若对数组用错 delete 而不是 delete[] ，只析构第一个元素或产生未定义行为；考试答案按页面显示的输出记忆，但真实 C++ 里要避免这种写法。

2018-2019 page 08

2018–2019 AS

Y¹/₂ True-or-False 10

A. Multiple-Choice – 1 10

A Fill-in-Blank 9

@ Fill-in-Blank – P 3

5–1 The function template printArrayInfo computes the minimal, maximal and average value and prints them out, where nrows is number of rows and ncols is the number of columns.

```
#include <iostream>
template<class T>
void printArrayInfo(T*
(15))
(11)) array, int nrows, int ncols)
(111) max = array[0], min = array[0];
double avg = 0
(155);
for (int i = 0; i < nrows; ++i)
for(int j = 0; j < ncols; ++j)
T val
if (val<min
if (val>max
avg =avg+static_cast<double>(vi (155);
(155) = array [i*ncols+j]
(117)) min = val;
(151)) max = val;
(15)) ;
Author: hulanqing
Organization: It
Time Limit: 400 ms
Memory Limit: 64 MB
avg /= (nrows*ncols
(15))) ;
std:: cout << "min=" << min << std::endl;
std:: cout << "max=" << max << std::endl;
std:: cout << "avg=" << avg << std::endl;
int main ()
int ai[2][3]={8,10,2}, (14,4,6}};
printArrayInfo (ai[0], 2, 3);
double af[1][5]={3.4f,4.2f,6.6f,2.4f, -0.9f} };
printArrayInfo(af[0], 1, 5);
return 0;
5–1 Accepted (10 points))
```

5-2 The class String is a simple C++ encapsulation of the C character arrays.

```
#include <cstring>
#include <iostream>
#include <stdexcept>
class stringIndexError : public std::out_of_range {
private:
    int index;
public:
    stringIndexError (int idx) : std::out_of_range(""), index (idx) {}
    int getIndex () const
    return index;
};
class string (
private:
    char *m_ptr;
public:
    String(const char *ptr)
    m_ptr = new char[strlen(ptr)+1]
    strcpy(m_ptr, ptr);
    (1t);
    ~String()
    delete[] m_ptr
    (15)) ;
    String &operator+=(const String &str)
    char *s = new char[strlen(m_ptr)+strlen(str.m_ptr)+1]
    (1):
    Author: hulanqing
    Organization: #FIX+
    Time Limit: 400 ms
    Memory Limit: 64 MB
```

解析：

本页是程序填空。printArrayInfo 考二维数组按连续内存传入、模板类型 T、min/max/avg 初始化与遍历；String 类考动态字符数组、析构 delete[] 、 operator+= 重新分配并拼接、下标越界抛异常、输出运算符 friend。

```

if (m_ptr)
strcpy(s, m_ptr);
delete[]
}
m_ptr
return *this
strcat(s, str.m_ptr); // appends str.m_ptr to s
bool operator== (const string &str) const
return (strcmp(m_ptr, str.m_ptr) == 0);
char& operator[] (int i)
if (i >= 0 && i < strlen(m_ptr)) return m_ptr[i];
throw StringIndexError (1);
friend
(157) m_ptr;
(15)) = s;
(155);
(111) std::ostream& operator<<(std::ostream &,
const string &);
std::ostream&
(15)) operator<< (std::ostream &out, const string &str)
return out << str.m_ptr;
int main()
String s1("Hello "), s2("world!");
if (s1 == s2)
std::cout << "S1==S2" << std::endl;
else
std::cout << "S1!=S2" << std::endl;
s1 += s2;
std::cout << s1 << std::endl;
try
(155) {
int k = 0;
while (true)
std::cout << s1[k++];
catch
(11) (const StringIndexError& ex) {
std::cout << "Instring index is out of range: " << ex.getIndex() << std::endl;
return 0;
5-2 Accepted (10 points))
5-3 The class Queue implements a circular queue data structure.
#include <iostream>

```

```

template<class T>
class Queue {
private:
int capacity;
/ capacity of the queue
(19)) data;
/ head of the queue
// tail of the queue
I/ dynamically allocated array of doubles
int front;
int rear;
public:
Queue (int maxsize);
~Queue (;
bool empty ();
bool full);
void push(Ta);
T pop();
// append a double value to the tail of queue
// delete the head element of the queue
template<class T> Queue<T>: :Queue (int maxsize)
capacity = maxsize;
data = new T[maxsize]
(15)) ;
Author: hulanging
Organization: #FIIt·
Time Limit: 400 ms
Memory Limit: 64 MB

```

解析：

本页延续 String 类。核心是深拷贝/拼接后更新 m_ptr，operator[] 做边界检查，越界时抛自定义异常，catch 中输出错误下标。OCR 把 String、strcpy、std:: 等符号识别得不稳定，代码以原卷为准。

```
front = rear = 0;
std:: cout << "queue initialized! ";
template<class T> Queue<T>::~~Queue()
delete[] data
std: : cout << "queue destroyed! ";
template<class T> bool Queue<T>: :empty()
return (front == rear)
template<class T> bool Queue<T>: : full ()
return (front == ((rear+1)%capacity))
(175);
(155) ;
(121);
//The dynamic array data will be a circular Queue
template<class T> void Queue<T>::push (T a)
if (full))
exit (0);
else
data[rear]
rear = (rear+1)%capacity
(11)) = a;
(155) ;
template<class T> T Queue<T>: : pop()
if (empty())
exit (0);
T top=data[front]
front = (front+1)%capacity
return top;
(155);
(145);
int main ()
Queue<double>
std:: cout << q. empty();
q. push (1.3);
q. push (2.3);
q. push (3.3);
q. push (4.3);
std: : cout << q.full();
q. pop () ;
q. pop ();
q. pop ();
q. push (5.3);
```

```
9. push (6.3);
q. push(7.3);
std:: cout << q.full();
q. pop ();
q. pop ();
q. pop ();
q. pop ();
std:: cout << q. empty();
return 0;
(157) q(5);
5-3 Accepted (10 points))
```

解析：

本页是循环队列程序填空。关键公式：空队列 $\text{front} == \text{rear}$ ，满队列

$(\text{rear} + 1) \% \text{capacity} == \text{front}$ ，入队写 $\text{data}[\text{rear}]$ 后 rear 前进，出队读 $\text{data}[\text{front}]$ 后 front 前进，数组用 $\text{new T}[\text{maxsize}]$ 和 delete [] data 。

2019-2020 春夏：Queue 大题（OCR 摘录）

8-1 Queue (355))

Design a generic circular queue class. using C++ standard exception class: overflow_err design.

the picture below shows the UML class diagram for the generic queue class.

-queueSize: const int

-head:

int

-rear:

-data buff:

+CQueue():

+(Queue(int s).

""the queue size, is a sens member, can store up to queuesize- 1 datas" »

*Record the subscript of the queue head"/.

"*Record the subscript of the end of the queue*/

I*Data storage buffer*/

/*the default queue size is 10*/

+getSize():

+getNumbers):

int const

int const

/*mean: the function return int, is a const member function*/+

/* Calculation formula for the total number of queue elements is : (rear - head + queueSize)%queueSize */+

+getHead():

+ deQueue():

+isEmpty():

+isFull):

+show():

bool const-

bool const

void const

The test code is in test.cpp, The ouput is (There is a space at the end of the line): n
05678

1F.

Figure 1 rear==head, The queue is empty+

Figure 2 Add elements' A'and' B to the queue in turn+

rear

Figure 3 Put the element'A' of the head out of the queue»

head

Figure 4 The queue is full when (rear+ 1) % QueueSize == front-

/*test. cpp*/

```

#include < iostream>
using namespace std;
#include "CQueue.h"
int main()
try {
Queue double rai
for (int i = 0; i < rq.getSize() - 1; i++)
if (rq. isFull()) printf( "now the queue is full! ");
if (!rq. isEmpty()) rq.show();
cout << ra. gethead() << endl;
for (int i = 0; i < 5; i++) // dequeueing 5 elements
rq. show() ;
catch (overflow_error& r)
cout << r.what() << endl;
catch (underflow_error& r)
cout << r.what() << endl;
return 0;
}
(355)

```

解析：

这是 19-20 的泛型循环队列大题。实现时要抓住四个点： `queueSize` 是 `const` 成员，必须在初始化列表初始化；队列最多存 `queueSize - 1` 个元素，用一个空位区分满和空；元素个数公式是 $(rear - head + queueSize) \% queueSize$ ；入队满时抛 `overflow_error`，出队空时抛 `underflow_error`。 `show()` 从 `head` 开始循环输出到 `rear` 前一个位置。

2019-2020 春夏：代码输出题（裁剪 OCR 摘录）

说明：原 PDF 为一张很长的图片页，已按纵向裁剪 OCR；可读性明显好于整页 OCR，但仍建议对照原 PDF。

===== /private/tmp/oop_ocr_pages/fmkraki5_crops/crop_01.png =====

4-1 The output of the code below is:

```
#include <cstring>
#include <iostream>
using namespace std;
class Stri
char m_s[10];
char *m_p;
public:
Str(char *s) (strcpy(m_s, s) ;m_p = m_s;}
operator char*() ( return m_p; }
char *operator++()
return m_p; }
char operator [] (Int 1) ( return m_s[1]; }
int main() (
Str s("H1");
cout << *s << endl;
++s;
cout << s[0] << endl;
cout < *s < endl;
```

The 1st line is

The 2nd line is •

The 3rd line is I

(155)

(15)

(15)

4-1

4-2 The output of the code below is:

```
#include <iostream>
using namespace std;
class Af
public:
A(){cout << "A()" << endl;}
~A(){cout << "~A()" < endl;}
class B : public A(
public:
B() {cout << "B()" << endl;}
~B () {cout << "~B)" << endl;}
};
```

91Z

1EM

9107

===== /private/tmp/oop_ocr_pages/fmkraki5_crops/crop_02.png =====

Puddy

```
B() {cout << "B()" << endl;}
~B() {cout << "~B()" << endl;}
};
```

```
int main()
```

The 1st line is

The 2nd line is

The 3rd line is

The 4th line is

The 5th line is

The 6th line is

(155)

(15)

(155)

(15)

(15)

(15)

4-2

4-3 The output of the code below is:

```
#include <iostream>
using namespace std;
template <typename T>
class FF(
    T a1, a2, a3;
public:
    ()
    T Sum() const
    FF (T b1, T b2, T b3): a1 (b1), a2 (b2), a3 (b3)
    return a1 + a2 + a3;
```

49-12

```
};
int main()
FF<int> x (2,3,4),y(-2, -3, -4) ;
cout << x.Sum() << endl << y.Sum() << endl;
```

The 1st line is 9

1(15)

The 2nd line is -9|

| (15)

4-4 The output of the code below is:

```
*include tinctnoams
struct A (
A(const A &a) { std: :cout << "B" << std: :endl; }
A& operator-(const A &a) {std::cout << "C" << std::endl; return *this;
```

===== /private/tmp/oop_ocr_pages/fmkraki5_crops/crop_03.png =====

```
A (const A da) i sta::cout < B < sta: :enal; }
```

```
A& operator= (const A &a) { std: cout << "C" << std: :endl; return *this; }
```

```
int main() (
```

```
A a[2];
```

```
A b = a [0];
```

```
A c;
```

```
C = a [1];
```

The 1st line is

The 2nd line is

The 3rd line is

The 4th line is

The 5th line is

(15)

(15)

(155)

(155)

(155)

4-4

4-5 The output of the code below is:

```
#include <iostream>
```

```
class C (
```

```
public:
```

```
explicit C(int) (
```

```
std: : cout << "1" << std: end];
```

```
C(double) {
```

```
std:: cout << "d" << std: :endl;
```

```
fEES
```

```
};
```

```
int main() (
```

```
C C1(7);
```

```
C c2 = 7;
```

The 1st line: i

15)|

The 2nd line: d

(13)

45 2)

4-6 The output of the code below is:

```
#include <iostream>
```

```
using namespace std;
```

```
class A{
```

```
public:
```

```
void F (double){ cout << "A: :F(double)" << endl; }
```

99107

```
class B: public A{
```

```
public:
```

```
void F(double){
```

```
cout << "B::F (double)" << endl; }
```

```
#I**
```

```
===== /private/tmp/oop_ocr_pages/fmkraki5_crops/crop_04.png =====
```

```
class B: public A{
```

```
public:
```

```
void F(double) {
```

```
cout << "B::F (double)" << endl; }
```

```
int main(){
```

```
B b;
```

```
b.F (2.0) ;
```

```
b.F (2) ;
```

```
b.F2 (2) ;
```

The 1st line is

The 2nd line is

The 3rd line is

(155)

(15)

(15)

4-6

4-7 The output of the code below is:

```
#include < iostream>
```

```
struct A {
```

```
virtual void foo(int a = 1) {
```

```
std: : cout << "A" << "In' << a;
```

```
} ;
```

```
struct B : A {
```

```
virtual void foo (int a = 2) {
```

```
std: : cout << "B" << "In' << a;
```

```
};
```

```
int main () {
```

```
A *a = new B;
```

```
a->foo () ;
```

The 1st line is

The 2nd line is I

(15)

(15)|

4-7

4-8 The output of the code below is:

```
*inclino tinctnoamy
```

```

using namespace std;
template<typename T>
T func (T x, double y){
return x+y;
int main(){
cout << func(2.7,3) << endl;
cout << func(3,2.7) << endl;
99-102
9940
===== /private/tmp/oop_ocr_pages/fmkraki5_crops/crop_05.png =====
cout fun(3,2.7) << endl;
The 1st line is 5.7
The 2nd line is 5
1(25)
(15)
4-9 The output of the code below is:
#include <iostream>
template<class T> void f(T &i) { std::cout << 1; }
template< void f(const int &i) < std::cout << 2: "
int main() (
int i = 24;
f (1) ;
The output is 1
(15)
49 7 1)
4-10 The output of the code below is:
zonaluab closurdamy
using namespace std;
class Af
int s[10];
public:
int operator[](int i) const
cout << "operator[] (int)const" << endl;return s[i];
int &operator[](int i){
cout << "operator[] (int) " << endl;return s[i];
};
int main(){
A a1;
const a 8a) = a1:
a1[0] = a2[1];
The 1st line is operator (int)const
The 2nd line is operator[](int)
4-10 *816 (24)

```

4-11 The output of the code below is:

```
#include <iostream>
using namespace std;
class Af
public:
static void f(double)(
cout << "f(double)" << endl;
(15)
(15)
4112
fEES
4112
Mias
*I**
===== /private/tmp/oop_ocr_pages/fmkraki5_crops/crop_06.png =====
public:
static void f (double)(
cout << "f (double)" << endl;
void f(int)(
cout << "f (int)" << endl;
int main()
const A a;
a.f(3);
```

The output is

(15)

4-11

4-12 The output of the code below is:

```
using namespace stai
class Al
public:
virtual ~A>()
} ;
class B : public A();
int main()
A a;
B b;
A *ap = &a;
if (dynamic_cast<B*>(ap))
cout << "OK1" << endl;
else
cout << "FAIL" << endl;
if (static_cast<B *>(ap)) |
cout < "OK2" << endl;
```

```

else
cout << "FAIL" << endl;
ap = 8b;
if (dynamic_cast<B*>(ap))
else
cout << "FAIL" << endl;
if (static_cast<B*>(ap))
cout << "OK4" << endl;
else
cout << "FAIL" << endl;
The 1st line is
The 2nd line is
The 3rd line is
(15)
(155)
|(15)
===== /private/tmp/oop_ocr_pages/fmkraki5_crops/crop_07.png =====

```

The 2nd line is i

The 3rd line is

The 4th line is

(155)

(15)

(15)/

4-12

4-13 The output of the code below is:

```

#include <iostream>
using namespace std;
class A1
static int m;
int n;
public:
A(int m,int n)(
this->m = m;
this->n = n;
void print()
cout << m << "... " << n << endl;
};
int A: :m;
a2. print();
The 1st line is
The 2nd line is
(255)
(15)

```

4-13

4-14 The output of the code below is:

```
*ncluee costreams
using namespace std;
class Af
public:
A(){ cout << "A()" << endl;}
A(const A&){ cout << "A(const A&) " << endl;}
A &operator-(const A&){
cout << "operator-(const A&) " << endl;
return *this;
}
#122
49102
#It*
===== /private/tmp/oop_ocr_pages/fmkraki5_crops/crop_08.png =====
```

a2 - a1;

A a3 - a2;

The 1st line is

The 2nd line is

The 3rd line is

The 4th line is

4-14

4-15 The output of the code below is:

```
sinclude costreams.
struct Base
virtual ~ Base()
std: : cout < "Destructing Base" < std: :endl;
Virtual void f()
std:: cout << "I'm in Base" << std:: endl;
struct Derived i public Base
~Derived()
std: : cout << "Destructing Derived" << std:: endl;
void f()
std:: cout << "I'm in Derived" << std: :endl;
};
```

The 1st line:

The 2nd line:

The 3rd line:

The 4th line:

4-15

解析（可识别题目）：

- 4-1: 考类型转换运算符、前置 ++ 和 operator[] 。先看 operator char*() 让对象可当字符指针用, 再看 ++s 是否移动内部指针, 最后用下标访问字符数组。
- 4-2: 考继承构造/析构顺序。创建 B 先 A 后 B; 销毁 B 先 B 后 A; 若有动态对象或作用域嵌套, 再按生命周期补输出。
- 4-3: 模板类 FF<T> 的 Sum() , int 版本直接把三个成员相加, 所以 2+3+4=9, -2-3-4=-9。
- 4-4: 考默认构造、拷贝构造和赋值运算符。A b = a[0] 是拷贝构造; c = a[1] 是赋值运算符, 不是拷贝构造。
- 4-5: explicit C(int) 只能直接初始化 C c1(7); C c2 = 7 不能用 explicit 的 int 构造, 若存在 C(double) 则通过转换走 double 构造。
- 4-6: 派生类 B 定义同名 F(double) 会隐藏基类同名重载; 如果调用不存在的 F2 或未引入基类重载, 按编译错误/名称隐藏规则判断。
- 4-7: 虚函数动态绑定看实际对象类型, 但默认参数静态绑定看指针静态类型; A* a = new B; a->foo() 调 B::foo, 默认参数仍取 A 中的 1。
- 4-8: 模板参数 T 由第一个参数推导。func(2.7,3) 中 T 为 double, 结果 5.7; func(3,2.7) 中 T 为 int, 返回值转 int, 所以为 5。
- 4-9: 函数模板重载/特化考引用匹配。普通 f(T&) 不能绑定右值 1, 若有 const int& 版本则可绑定; 按可行函数和更特化规则选。
- 4-10: const 对象只能调用 const 版本 operator[] , 非 const 对象赋值左侧调用非 const 版本; 表达式求值顺序也会影响两行输出顺序, 考试按原卷答案记。
- 4-11: const 对象不能调用非 const 成员函数; static 成员函数没有 this, 可通过对象调用但本质属于类。
- 4-12: dynamic_cast 对指针失败返回 nullptr; static_cast 不做运行时检查, 只要编译允许就可能得到非空指针, 但不代表对象真实类型正确。
- 4-13: static 成员 m 被所有对象共享, this->m = m 修改的是类共享变量; 普通成员 n 每个对象各有一份。
- 4-14: a2 = a1 是赋值运算符; A a3 = a2 是拷贝构造。看清“创建新对象”和“已有对象赋值”的区别。
- 4-15: 基类析构 virtual 时, 通过 Base* 删除 Derived 对象会先析构 Derived 再析构 Base; 虚函数 f() 通过基类指针调用时按实际对象类型输出 Derived 版本。